



JDBC事务处理

北京理工大学计算机学院
金旭亮

理解“数据库事务”的基本特性



如果有若干条SQL语句需要执行，它们要么统统成功，要么完全失败，不会出现部分成功的情况。满足这个条件的这些SQL语句，就构成了一个事务：transaction。



如果SQL操作全部成功，就调用commit方法提交事务。



如果至少有一个SQL操作失败，就调用rollback方法进行“回滚”，恢复数据库到执行前的状态，以保证数据的一致性和有效性。

数据库事务处理的四个特性



1

Atomicity (原子性)：如果需要对数据库执行多个修改命令，那么这个操作要不全部成功，要不维持原状。

2

Consistency (一致性)：事务执行前后，数据库中的数据都应该都处于一种有效的状态之中。

3

Isolation (隔离性)：不同事务之间，不互相影响。在一个事务没有完成之前，其状态是不可见的，外界无法干涉。

4

Durability (持久性)：事务一旦完成，它对数据库的修改就是持久有效的。即使后来数据库马上崩溃，重启之后，这些修改仍然存在。

JDBC提交修改的两种模式



自动模式下，所有sql语句正常执行，每句一个事务，这是系统默认模式。

当语句返回ResultSet的时候，事务结束的时机要不是最后一行ResultSet返回出来的时候，就是ResultSet关闭的时候。



在非自动模式下，可以将多条SQL语句组成一个组，完成对数据库的多个操作，最后再提交，这时，每个组对应一个事务。

事务执行与否取决于执行以下哪个方法：`commit` 还是 `rollback`

事务处理基本代码框架



先用`conn.setAutoCommit(false)`禁止自动提交事务，操作完成后调用`conn.commit()`整体提交，若一条SQL失败，产生`SQLException`，则在`catch`块中使用`conn.rollback()`撤消事务。

```
Connection conn =...;

//关闭自动事务特性
conn.setAutoCommit(false);

try{
    //修改数据库
    conn.commit();
}
catch(SQLException){
    //回滚
    conn.rollback();
}
```

定义数据库事务的基本操作-删除与查询

//删除所有记录

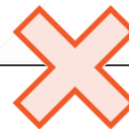
```
private static void deleteAllRecords(Statement statement)
    throws SQLException {
    String sql = "delete from OrderClient";
    int rowAffected = statement.executeUpdate(sql);
    System.out.println("数据删除条数:" + rowAffected);
}
```



删除会修改数据库的状态，因此是事务的监控对象（打勾），而查询是只读的，不会对事务有任何的影响（打叉）。

//显示数据库中的记录数

```
private static void showCountOfRecord(Statement statement)
    throws SQLException {
    String sql = "select count(*) as count from OrderClient";
    var rs : ResultSet = statement.executeQuery(sql);
    rs.next();
    System.out.println("当前数据库中记录条数: " + rs.getInt(columnLabel: "count"));
}
```



定义数据库的基本操作-插入



//插入一条记录

```
private static void insert(OrderClient orderClient, Statement statement)
    throws SQLException {
    String sql = "insert into OrderClient(ClientName,Address) values('"
        + orderClient.getClientName() + "','"
        + orderClient.getAddress() + "')";
    int rowAffected = statement.executeUpdate(sql);
    if (rowAffected > 0) {
        System.out.println("数据已经成功插入:"+orderClient);
    } else {
        System.out.println("插入操作未成功");
    }
}
```


成功完成的事务

```
static void testSucceedTransaction() {  
    try (Connection connection = DriverManager.getConnection(dbUrl);  
        Statement statement = connection.createStatement();  
    ) {  
        showCountOfRecord(statement); //显示记录数  
        connection.setAutoCommit(false); //指明需手动执行事务  
        try{  
            deleteAllRecords(statement); //删除全部数据  
            //插入5条记录  
            for (int i = 0; i < 5; i++) {  
                insert(OrderClient.getInstance(), statement);  
            }  
            connection.commit();  
            System.out.println("操作结束");  
        }  
    }  
}
```

```
        catch (Exception e){  
            System.out.println(e.getMessage());  
            connection.rollback();  
            System.out.println("回滚");  
        } finally {  
            //显示记录数  
            showCountOfRecord(statement);  
        }  
    } catch (SQLException e) {  
        System.err.println(e.getMessage());  
    }  
}
```


运行截图



```
Run: Main x
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\
当前数据库中记录条数: 5
数据删除条数: 5
数据已成功插入:OrderClient{clientName='client58', address='address of client58'}
数据已成功插入:OrderClient{clientName='client71', address='address of client71'}
数据已成功插入:OrderClient{clientName='client52', address='address of client52'}
数据已成功插入:OrderClient{clientName='client60', address='address of client60'}
数据已成功插入:OrderClient{clientName='client85', address='address of client85'}
操作结束
当前数据库中记录条数: 5

Process finished with exit code 0
```

不成功的事务

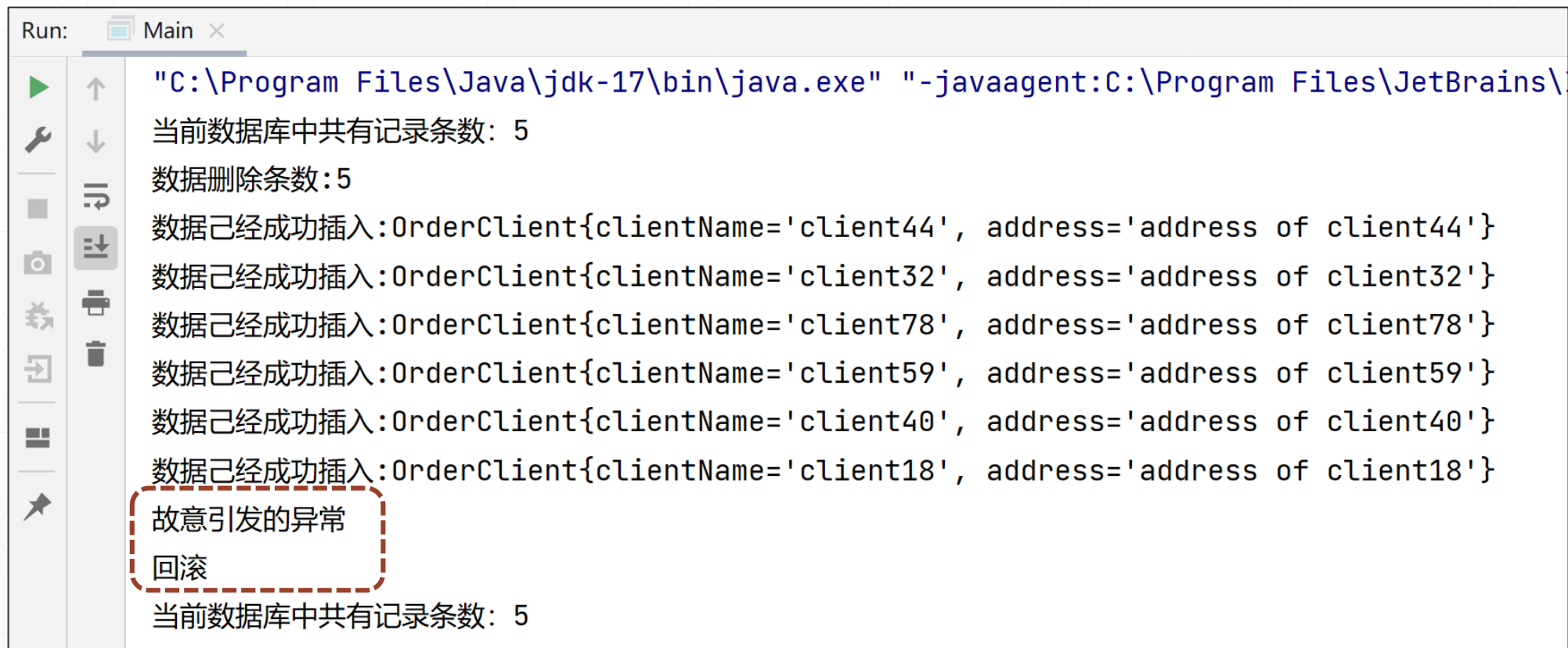
```
static void testFailedTransaction() {  
    try (Connection connection = DriverManager.getConnection(dbUrl);  
        Statement statement = connection.createStatement();  
    ) {  
        showCountOfRecord(statement); //显示记录数  
        connection.setAutoCommit(false); //指明需手动执行事务  
        try {  
            //删除全部数据  
            deleteAllRecords(statement);  
            //插入10条记录  
            for (int i = 0; i < 10; i++) {  
                insert(OrderClient.getInstance(), statement);  
                if (i == 5) {  
                    throw new RuntimeException("故意引发的异常");  
                }  
            }  
            connection.commit();  
            System.out.println("操作结束");  
        }  
    }  
}
```



由于在插入记录过程中出现了异常，因此，整个事务被回滚.....

```
        catch (Exception e) {  
            System.out.println(e.getMessage());  
            connection.rollback();  
            System.out.println("回滚");  
        } finally {  
            //显示记录数  
            showCountOfRecord(statement);  
        }  
    } catch (SQLException e) {  
        System.err.println(e.getMessage());  
    }  
}
```

运行截图



The screenshot shows a Java IDE's Run console window. The title bar says "Run: Main x". The console output is as follows:

```
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\Idea\bin\jetbrains-agent.jar" -jar ...  
当前数据库中共有记录条数: 5  
数据删除条数: 5  
数据已经成功插入:OrderClient{clientName='client44', address='address of client44'}  
数据已经成功插入:OrderClient{clientName='client32', address='address of client32'}  
数据已经成功插入:OrderClient{clientName='client78', address='address of client78'}  
数据已经成功插入:OrderClient{clientName='client59', address='address of client59'}  
数据已经成功插入:OrderClient{clientName='client40', address='address of client40'}  
数据已经成功插入:OrderClient{clientName='client18', address='address of client18'}  
故意引发的异常  
回滚  
当前数据库中共有记录条数: 5
```

The text "故意引发的异常" (Intentionally triggered exception) and "回滚" (Rollback) are enclosed in a red dashed box.

请在执行前后查看数据库中的记录，你会发现，执行前后，数据库中的记录保持原样，代码中的修改没有生效。